SOFTWARE ENGINEERING – LAB 7

Fuzz Testing for Input Validation Bugs

Student Name	Charan
SRN	PES1UG25CS856
Repository	SETAPESU26 / L6_L7_64_bowling_alley
Folder	LAB_7
Testing Library	Hypothesis with pytest

Submission evidence: original run, fuzz-test failures, corrected run, and passing fuzz tests.

SS1 – Original Buggy Application

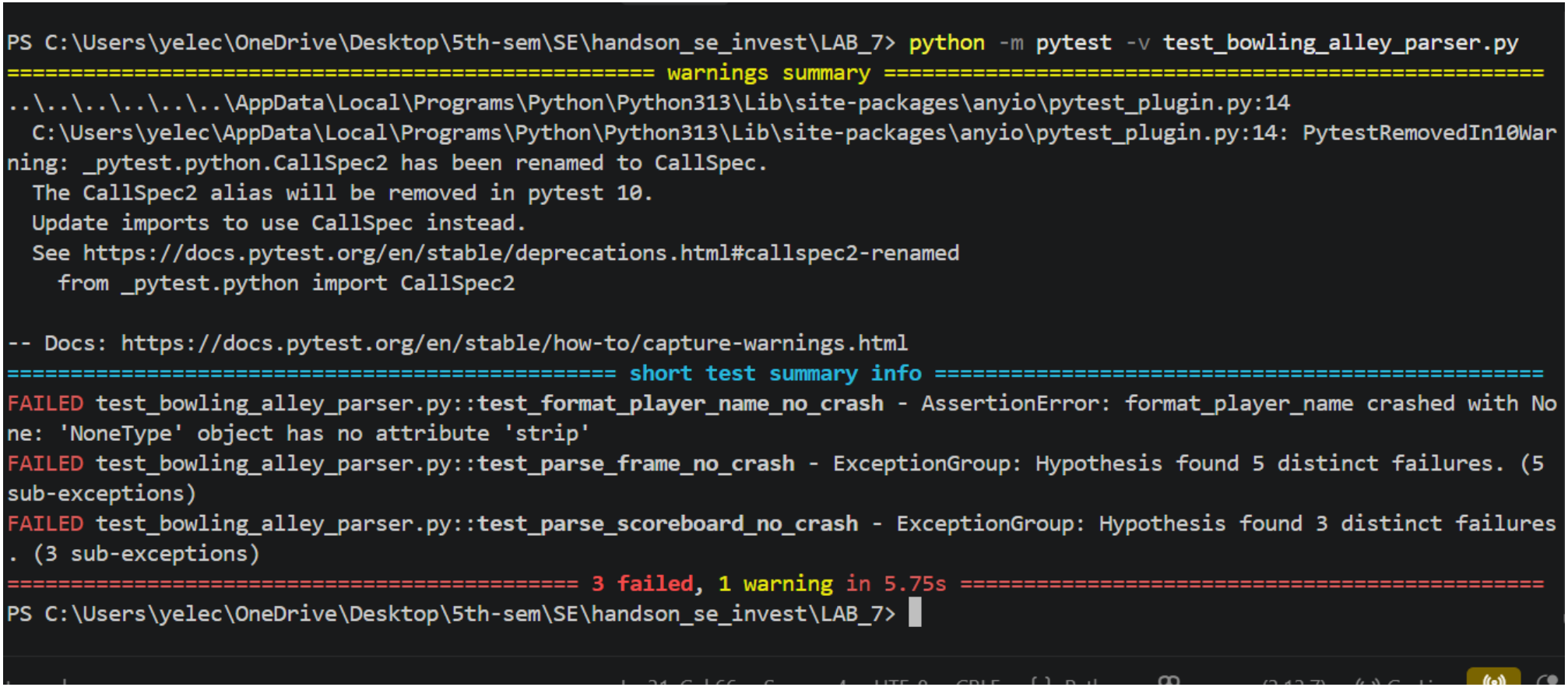
```
PS C:\Users\yelec\OneDrive\Desktop\5th-sem\SE\handson_se_invest\LAB_7> python bowling_alley_parser.py
==== Running format_player_name ====
Enter a player name for the scoreboard: charan
Scoreboard name: Charan

==== Running parse_frame ====
Enter a frame as X (strike), 7/ (spare) or two digits (e.g. 45): 7/
Pins (first roll, second roll): (7, 3)

==== Running parse_scoreboard ====
Enter scores as NAME:SCORE separated by commas (e.g. Asha:150,Ravi:132): Asha:150,Ravi:132
Scoreboard: {'Asha': 150, 'Ravi': 132}
PS C:\Users\yelec\OneDrive\Desktop\5th-sem\SE\handson_se_invest\LAB_7>
```

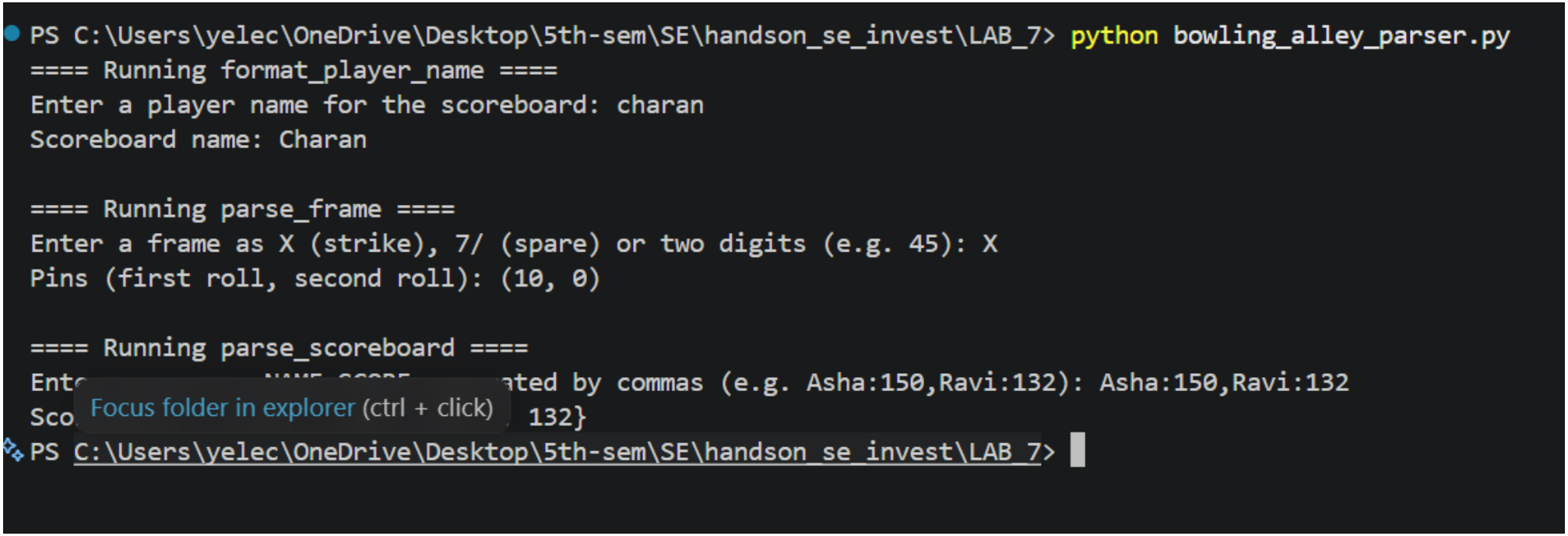
The original application ran successfully with valid sample inputs.

SS2 – Hypothesis Tests on Original Application



Hypothesis exposed failures in all three functions when it generated unexpected inputs.

SS3 – Fixed Application



The corrected application was run with a player name, a strike frame, and scoreboard entries.

SS4 – Hypothesis Tests After Fixing

```
PS C:\Users\yelec\OneDrive\Desktop\5th-sem\SE\handson_se_invest\LAB_7> python -m pytest -v test_bowling_alley_parser.py
plugins: anyio-4.13.0, hypothesis-6.170.15
collected 3 items

test_bowling_alley_parser.py::test_format_player_name_no_crash PASSED                [ 33%]
test_bowling_alley_parser.py::test_parse_frame_no_crash PASSED                       [ 66%]
test_bowling_alley_parser.py::test_parse_scoreboard_no_crash PASSED                  [100%]

============================================== warnings summary ==============================================
..\..\..\..\..\..\AppData\Local\Programs\Python\Python313\Lib\site-packages\anyio\pytest_plugin.py:14
  C:\Users\yelec\AppData\Local\Programs\Python\Python313\Lib\site-packages\anyio\pytest_plugin.py:14: PytestRemovedIn10War
ning: _pytest.python.CallSpec2 has been renamed to CallSpec.
  The CallSpec2 alias will be removed in pytest 10.
  Update imports to use CallSpec instead.
  See https://docs.pytest.org/en/stable/deprecations.html#callspec2-renamed
    from _pytest.python import CallSpec2

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================== 3 passed, 1 warning in 1.33s ==========================================
PS C:\Users\yelec\OneDrive\Desktop\5th-sem\SE\handson_se_invest\LAB_7>
```

All three no-crash property tests passed after input validation was added.

Reflection

Note: The supplied lab handout lists the screenshot and code deliverables but does not show the wording of three reflection questions. The following are concise reflections based on the work performed in this lab.

1. What kinds of bugs did fuzz testing reveal?

Hypothesis generated unexpected values such as None, empty strings, short frame strings, invalid characters, and malformed scoreboard entries. These inputs exposed AttributeError, IndexError, and ValueError crashes in the original functions.

2. How did Hypothesis help identify the bugs?

Instead of relying only on a few manually selected examples, Hypothesis generated many inputs from the text and None strategies. When a failure occurred, it reported a minimal failing example, such as None, an empty string, or a short invalid frame. This made the problems easier to reproduce.

3. What changes made the application more robust?

The functions now validate input types and structure before processing. Invalid player-name inputs return an empty string, invalid frame notation returns (0, 0), and malformed scoreboard entries are skipped. After these changes, all three Hypothesis no-crash tests passed.

Test result: 3 passed, 1 warning. The warning shown in the terminal is a pytest/AnyIO deprecation warning and is not a failure in the lab application.

Files to submit alongside this PDF: bowling_alley_parser.py and test_bowling_alley_parser.py.